

AddisCoder 2025: Week 2

AddisCoder 2025: Week 2

August 04, 2025

Taught by Huy Nguyen.

Start with slides!

Quiz Problem Review

The three most missed problems were #16, #17, and #18. Let's solve them together!

```
# Question #16:
var = 3
def myEquation(var):
    var *= 4
    var += 10
    return var

# 16a
print(myEquation(10))

50

# 16b
print(var)

3

# Question #17: deepenList.

def deepenList(lst):
    new_list = []
    for e in lst:
        new_list.append([e])
    return new_list

print(deepenList([1]))
print(deepenList([[1]]))
print(deepenList([1, 2]))
print(deepenList([]))

[[1]]
[[[1]]]
[[1], [2]]
[]
```

```
# Question #18: most_common(lst), elements_counts(lst)
def element_counts(lst):
    counts = {}
    for e in lst:
        if e in counts:
            counts[e] += 1
        else:
            counts[e] = 1
    return counts

def most_common(lst):
    counts = element_counts(lst)
    max_count = 0
    common_elt = None
    for key, value in counts.items():
        if value > max_count:
            max_count = value
            common_elt = key
    return common_elt

print(element_counts([1, 1, 2]))
print(element_counts(["a", "a", "b"]))
print(element_counts([]))

print(most_common([1, 2, 2]))
print(most_common(["a", "a", "b"]))
print(most_common([]))

{1: 2, 2: 1}
{'a': 2, 'b': 1}
{}
2
a
None
```

Dictionaries

Just like real-world dictionaries, these are used for *lookup*. You have some key (ex: a word) and you want to find the associated value (ex: a definition). Let's walk through [the slides](#) first.

```
# For creating dictionaries, use curly braces:
gradebook = {}
print(type(gradebook))
print(gradebook)

<class 'dict'>
{}
```

*# We add key:value pairs to our dictionary using square brackets on the *left* of the equals sign.*

```
print(gradebook)
print("Adding new key:value pair.")
gradebook["alex"] = 95
print(gradebook)
```

```
{}
```

Adding new key:value pair.
{'alex': 95}

*# You can get the value associated with a key by using brackets *not* to the left of an equal sign:*

```
print(gradebook["alex"])
```

95

What happens if you try to lookup a key that doesn't exist in the dictionary? Error!

```
print(gradebook)
print("Daniel's score:", gradebook["daniel"])
```

```
{'alex': 95}
```

```
-----
-----
KeyError                                Traceback (most recent call
last)
```

```
Cell In[13], line 3
```

```
    1 # What happens if you try to lookup a key that doesn't exist
in the dictionary? Error!
```

```
    2 print(gradebook)
```

```
----> 3 print("Daniel's score:", gradebook["daniel"])
```

```
KeyError: 'daniel'
```

To avoid this issue, we can check if a dictionary contains something using `in`

```
if "daniel" in gradebook:
    print("Daniel's score:", gradebook["daniel"])
else:
    print("daniel is not in the dictionary")
```

```
daniel is not in the dictionary
```

To iterate over dictionary keys, use the forloop syntax. It works for dictionaries automatically!

```

for key in gradebook:
    print("name:", key, ", grade:", gradebook[key])

name: alex , grade: 95

# Common pattern when building dictionaries as counters:

counter_dict = {}
word = "horse"
if word in counter_dict:
    counter_dict[word] = counter_dict[word] + 1
else:
    counter_dict[word] = 1
print(counter_dict)

{'horse': 1}

```

Function Trickiness

We saw some common mistakes with functions on the exam and when walking around during lab. Let's clarify them here.

```

# (1) Defining vs Calling Functions

# Defining a function:
def invert_string(s):
    new_string = ""
    for i in range(len(s) - 1, -1, -1):
        new_string = new_string + s[i]
    return new_string

# Running this block creates the function, but nothing actually gets
run because the function hasn't been called.

# Calling a function:
print(invert_string("hello"))

# Running this block actually runs the function and prints the output.
olleh

# (2) Return vs Print

# An "expression" is any statement that evaluates to a value. For
example, `1 + 1` is an expression that evaluates to `2`.
# Functions are also expressions. For example, `len("hello")` is an
expression that evaluates to `5`.

def fun_func():
    return 5

```

The `return` statement tells Python what value the function should evaluate to. In this case, the function evaluates to `5`.

```
print(fun_func())
```

5

Not all functions evaluate to something! You can tell because they have no `return` statement. For example:

```
def void_func():  
    x = 1 + 1
```

This function doesn't return anything. It just runs some code. If we try to print the output of this function, we get `None`.

```
print(void_func())
```

None

void_func() isn't very useful, but the more common case is when you want to print something inside a function. For example:

```
def print_func():  
    print("hello")
```

This function doesn't return anything. It just prints something. If we try to print the output of this function, we get `None`.

```
print(print_func())
```

hello

None

So, we use `print` to print things to the console, and we use `return` to return values from functions.

So often times, there's two ways to write a function if you want to show something on

the screen: one that returns a value, and one that prints a value.

```
def return_sum(a, b):  
    return a + b  
print(return_sum(1, 2))
```

3

```
def print_sum(a, b):  
    print(a + b)  
print_sum(1, 2)
```

3

```

# Notice: these both do the same thing in the end, but in different
ways.

# (3) function decomposition: breaking larger problems down in a
collection of smaller functions

# Example: write a function that takes a string and returns the number
of vowels in the string.

# Step 1: write a function that takes a string and returns True if the
string is a vowel, False otherwise.
def is_vowel(char):
    if char == "a" or char == "e" or char == "i" or char == "o" or
char == "u":
        return True
    else:
        return False

# Step 2: write a function that takes a string and returns the number
of vowels in the string.
def count_vowels(s):
    num_vowels = 0
    for char in s:
        if is_vowel(char):
            num_vowels += 1
    return num_vowels

# Step 3: test your function
print(count_vowels("hello"))

2

```

Default Parameters

```

# We've learned how to specify the arguments that a function should
take:

def greet(name, greeting):
    print(greeting + " to you, " + name + "!")

greet("Alex", "Hello")

Hello to you, Alex!

# In cases where you use the same argument value over and over, it can
get annoying to type each time.
greet("Daniel", "Hello")
greet("Huy", "Hello")
greet("Huacheng", "Hello")
greet("Jelani", "Selam")

```

```
Hello to you, Daniel!  
Hello to you, Huy!  
Hello to you, Huacheng!  
Selam to you, Jelani!
```

*# Python allows us to specify a _default value_ for an argument.
If the caller doesn't specify a value for that argument, the default value is used instead.*

```
def greet(name, greeting="Hello"):  
    print(greeting + " to you, " + name + "!")
```

```
greet("Daniel")  
greet("Huy")  
greet("Huacheng")
```

If the caller specifies a value for that argument, the default value is ignored.

```
greet("Jelani", "Selam")
```

```
Hello to you, Daniel!  
Hello to you, Huy!  
Hello to you, Huacheng!  
Selam to you, Jelani!
```